

Architettura degli Elaboratori

2022-2023

Indice:

Sistemi numerici:.....	6
Sistema numerico posizionale:.....	6
Sistema decimale:	6
Sistema binario:	6
Bit più e meno significativo:	6
Sistema ottale:.....	7
Sistema esadecimale:	7
Conversione tra sistemi numerici:	7
Overflow:	7
Rappresentazione numeri con segno:	8
Somma in binario:	8
Sottrazione in binario:	8
Rappresentazione dei numeri negativi:	8
Rappresentazione modulo e segno:	8
Rappresentazione complemento a 1:	9
Rappresentazione complemento a 2:	9
Operazioni aritmetiche in modulo e segno:	9
Sottrazione di due valori in modulo e segno:	10
Somma di due valori in CA2:	10
Sottrazione di due valori in CA2:	10
Rappresentazione Eccesso 128:	10
Operazione di shift:	11
Rappresentazione numeri reali:	11
Rappresentazione in virgola fissa:	11
Rappresentazione in virgola mobile:	11
Logica combinatoria:	12
Circuiti digitali:	12
Porta logica AND:	12
Porta logica OR:	13
Porta logica NOT:	13
Porte logiche derivate:	13

Porta logica NAND:	13
Porta logica NOR:	14
Porta logica XOR:	14
Decoder:	14
Multiplexer:.....	15
ALU (arithmetic logic unit):	15
Circuiti sequenziali:.....	16
Sr-latch:	16
Clock:	17
Latch di tipo D:	18
Metodologia di timing Edge-triggered:	18
Flip Flop:.....	18
Automati a stati finiti:	18
Registri:	19
Gerarchia delle memorie:.....	19
Memoria RAM (random access memory):	19
SRAM:	20
DRAM:	20
ISA (instruction set architecture):.....	20
Istruzione R-Type:	20
Istruzione I-Type (addition immediate):.....	21
Istruzione J-Type:	22
Programmazione Assembly:.....	22
Linker:	22
Loader:	22
Assembler:	23
Passaggio di parametri:.....	23
Registri temporary (\$t):	24
Registri saved (\$s):	24
Registri argomento(\$a):	24
Registri valore(\$v):	24
Stack:.....	24
Convenzione in assembly nomi e usi dei registri:.....	25

Datapath:	25
Datapath singolo ciclo:	25
Componenti del Datapath:	25
Passi per l'esecuzione di una istruzione:	26
Quali operazioni esegue un processore:	26
Implementazione della fase di Fetch:	26
Fase di Decode:	27
Execute R-Type:	27
Execute Beq:	27
Execute jump:	28
Implementazione Multi-ciclo:	28
Registri temporanei:	28
Eccezioni:	29
Registro cause:	29
Registro EPC:	30
Gestore delle eccezioni:	30
Indirizzo fisso:	30
Interruzioni vettorizzate:	30
I/O memory mapped:	31
Valutazione della prestazione:	31
Banda passante:	31
Latenza:	31
Controllo di programma:	31
I/O guidato da interrupt:	31
DMA – accesso diretto alla memoria	32
Controllo del percorso dei dati in pipelining:	32
Hazards:	32
Hazard strutturali:	32
Hazard sui dati:	32
Propagazione nel caso di due o più istruzioni:	33
Hazard sul controllo:	34
Cache:	34
Gerarchie di memoria:	34
Memoria cache:	34

Principio di località:	35
Tipologie di memoria cache:	35
Mappatura di un indirizzo:.....	36
Dimensione del blocco di cache:.....	36
Sostituzione dei blocchi:.....	36
Gestione delle miss in lettura:.....	36
Write miss:	37
Tips and tricks:.....	37

Sistemi numerici:

Con il termine Bit definiamo l'unità di misura dell'informazione, un bit può assumere solo due valori 1 e 0.

Una sequenza di bit può assumere 2^K configurazioni, ad esempio dati 4 bit la sequenza può assumere fino a 16 configurazioni in quanto $2^4 = 16$

Sistema numerico posizionale:

Un sistema numerico posizionale è un sistema in cui ciascuna cifra assume uno specifico significato in base alla posizione occupata nella sequenza di numeri.

Sistema decimale:

Il sistema decimale è caratterizzato da:

- Una base $r = 10$
- Un insieme di cifre comprese tra 0 e 9

Un numero n viene rappresentato secondo la seguente formula:

$$N = d_{n-1} * 10^{n-1} + \dots + d_0 * 10^0 + d_{-1} * 10^{-1} + \dots + d_{-m} * 10^{-m}$$

Sistema binario:

Il sistema binario è caratterizzato da:

- Una base $r = 2$
- Due sole cifre 0, 1

Un numero n viene rappresentato secondo la seguente formula:

$$N = d_{n-1} * 2^{n-1} + \dots + d_0 * 2^0 + d_{-1} * 2^{-1} + \dots + d_{-m} * 2^{-m}$$

Bit più e meno significativo:

In un byte, una sequenza di 8 bit consecutivi si possono individuare il bit più significativo e quello meno significativo:

11010010

La prima cifra evidenziata in rosso è detta (MSB) ovvero most significant bit.

L'ultima cifra evidenziata in blu è detta (LSB) ovvero least significant bit.

Sistema ottale:

Il sistema ottale è caratterizzato da:

- Una base $r = 8$
- Un insieme di cifre comprese tra 0 e 7

Un numero n viene rappresentato secondo la seguente formula:

$$N = d_{n-1} * 8^{n-1} + \dots + d_0 * 8^0 + d_{-1} * 8^{-1} + \dots + d_{-m} * 8^{-m}$$

Sistema esadecimale:

Il sistema esadecimale è caratterizzato da:

- Una base $r = 16$
- Un insieme di cifre comprese tra 0....9 con l'aggiunta di A,B,C,D,E,F che corrispondono ai numeri fino al 15.

Un numero n viene rappresentato secondo la seguente formula:

$$N = d_{n-1} * 16^{n-1} + \dots + d_0 * 16^0 + d_{-1} * 16^{-1} + \dots + d_{-m} * 16^{-m}$$

Conversione tra sistemi numerici:

La conversione da qualsiasi base r a base 10 avviene secondo la seguente formula:

$$N_r = d_{n-1} * r^{n-1} + \dots + d_0 * r^0 = M_10$$

La conversione da base 10 a qualsiasi base r invece rispetta i seguenti passaggi:

1. Dividiamo il valore numerico N per la base r fino a quando l'ultimo quoziente è minore della base stessa (r).
2. Prendiamo l'ultimo quoziente e tutti i resti delle divisioni, e procedendo dall'ultimo resto al primo, li scriviamo da sinistra verso destra.

La conversione da qualsiasi base p a qualsiasi base q generalmente passa attraverso la base 10.

- Convertire il numero da base p a base 10.
- Convertire il numero da base 10 a base q .

Overflow:

La rappresentabilità dei valori è legata al numero di cifre disponibili, si ha overflow quando non è possibile rappresentare un risultato di un'operazione con il numero di cifre a disposizione.

Rappresentazione numeri con segno:

Somma in binario:

Si esegue la somma tra bit di pari ordine secondo le seguenti regole:

1. $0 + 0 = 0$
2. $0 + 1 = 1$
3. $1 + 0 = 1$
4. $1 + 1 = 0$ con il riporto di 1 sul bit di ordine superiore.
5. $1 + 1 + 1 = 1$ con il riporto di 1 sul bit di ordine superiore.

Sottrazione in binario:

Si esegue la sottrazione tra bit di pari ordine secondo le seguenti regole:

1. $0 - 0 = 0$
2. $1 - 0 = 1$
3. $1 - 1 = 0$
4. $0 - 1 = 1$ con il prestito dal bit di ordine superiore.

Rappresentazione dei numeri negativi:

È possibile rappresentare i numeri negativi mediante quattro rappresentazioni differenti:

1. Modulo e segno (MS)
2. Complemento a 1 (CA1)
3. Complemento a 2 (CA2)
4. Eccesso 128

Rappresentazione modulo e segno:

Supponiamo di avere a disposizione 1 byte ovvero 8 bit per la rappresentazione di numeri positivi e negativi.

Ricorrendo all'utilizzo della rappresentazione in modulo e segno utilizzeremo:

- I primi 7 bit a destra per il valore assoluto del numero.
- Il bit più a destra, ovvero l'MSB per rappresentare il segno.

L'MSB assume due valori differenti, 0 quando il numero è positivo, 1 quando il numero è negativo.

Esempio:

Rappresentazione del numero -4 su 8 bit:

$$-4 \rightarrow 10000100$$

Il segno è rappresentato dal numero evidenziato in rosso, mentre le cifre evidenziate in blu rappresentano il valore assoluto.

Con la rappresentazione MS avendo a disposizione n bit totali si possono rappresentare i numeri interi nell'intervallo:

$$[-(2^{n-1} - 1), +(2^{n-1} - 1)]_{10}$$

Rappresentazione complemento a 1:

1. Se il numero da codificare è positivo, lo si converte in binario con il metodo tradizionale.
2. Se il numero da codificare è negativo, lo si converte in binario e successivamente si esegue l'operazione di complemento sulla codifica binaria effettuata.

Esempio:

Supponiamo di voler rappresentare il numero -3 su 4 bit in complemento a 1 (CA1)

- Il primo passo è convertire 3 in binario: $3_{10} = 0011_2$
- Una volta ottenuto il valore binario occorre eseguire la complementazione: $0011_2 = 1100_2$

Rappresentazione complemento a 2:

1. Se il numero da codificare è positivo allora rimane invariato
2. Se il numero da codificare è negativo:
 - a. Si effettua l'operazione di complemento a 1.
 - b. Si somma +1 al risultato in complemento a 1.

In CA2 i valori negativi hanno MSB = 1.

Dati n bit con la rappresentazione in CA2 si possono rappresentare i numeri nell'intervallo:

$$[-(2^{n-1}), +(2^{n-1} - 1)]_{10}$$

Come calcolare il CA2:

Per calcolare CA2 si calcola CA1 e si somma 1 quindi:

$$CA2(X) = CA1(X) + 1$$

Conversione da CA2 a decimale:

1. Se il numero è positivo (MSB = 0), si converte in base decimale usando il numero binario puro.
2. Se il numero è negativo (MSB = 1), si applica l'operazione di CA2 a questo valore ottenendo la rappresentazione del corrispondente positivo, si converte il risultato come numero in binario puro e si aggiunge il segno meno.

Operazioni aritmetiche in modulo e segno:

somma di due valori in modulo e segno

Confronto i bit di segno dei due numeri:

- Se i bit di segno sono uguali:
 - Il bit di segno risultante sarà il bit di segno dei due addendi.
 - Eseguo la somma bit a bit a meno di overflow.

- Se i bit di segno sono diversi:
 - Confronto i valori assoluti dei due addendi
 - Il bit di segno risultante sarà il bit di segno dell'addendo con valore assoluto maggiore.
 - Eseguo la differenza bit a bit.

Sottrazione di due valori in modulo e segno:

Confronto i bit di segno dei due numeri:

- Se i bit di segno sono uguali:
 - Il bit di segno risultante sarà uguale al bit di segno dell'operando a modulo maggiore.
 - Il risultato avrà modulo pari al modulo della differenza dei moduli degli operandi.
- Se i bit di segno sono diversi:
 - Il bit di segno risultante sarà uguale al bit di segno del minuendo.
 - Il risultato avrà modulo pari alla somma dei moduli dei due operandi.

$$A - B = A + (-B)$$

Somma di due valori in CA2:

1. Si esegue la somma su tutti i bit degli addendi, segno compreso.
2. Un eventuale riporto oltre il bit di segno viene scartato.
3. Nel caso in cui entrambi gli operandi siano di segno concorde è necessario verificare l'assenza di overflow.

Sottrazione di due valori in CA2:

La sottrazione tra due numeri in CA2 viene trasformata in somma applicando la regola seguente:

$$A - B = A + (-B)$$

Ovvero:

$$A - B = A + CA2(B)$$

Rappresentazione Eccesso 128:

Nella rappresentazione eccesso 128 un numero X viene rappresentato come segue:

$$X + 128$$

L'intervallo di rappresentazione dei valori è il seguente:

$$[-2^{n-1}, +2^{n-1} - 1]$$

Esempio:

$$x = 5 \quad x = 5 + 128 \rightarrow 133$$

$$x = -3 \quad x = -3 + 128 \rightarrow 125$$

Operazione di shift:

L'operazione di shift consiste nello spostare verso destra o verso sinistra la posizione delle cifre di un numero, espresso in una base qualsiasi, inserendo nelle posizioni lasciate libere uno 0. Esistono due tipi di shift:

1. Shift Left: equivale a moltiplicare il numero per la base.
2. Shift Right: equivale a dividere il numero per la base.

Rappresentazione numeri reali:

Rappresentazione in virgola fissa:

La rappresentazione di un numero decimale in virgola fissa avviene secondo i seguenti passaggi:

1. Si considera la parte intera del numero che si vuole rappresentare e la si codifica in binario.
2. La parte decimale del numero viene scomposta attraverso moltiplicazioni successive dalle quali si prenderà solo la parte intera.

Esempio:

Se voglio convertire il numero 21,25 in virgola fissa agisco nel seguente modo:

- Trasformo il numero 21 da decimale a binario $21_{10} = 10101_2$
- Successivamente mi occupo della parte decimale che viene scomposta come segue:
 $0,25 * 2 = 0,5 \rightarrow 0$
 $0,5 * 2 = 1 \rightarrow 1$
 $0,0 * 2 = 0$

Dai risultati delle moltiplicazioni estraggo solamente la parte intera che andrà a comporre la parte decimale del numero:

$$21,25_{10} = 10101,01_2$$

Rappresentazione in virgola mobile:

Precisione semplice:

Rappresentazione su 32bit - 4byte

segno	Esponente	Mantissa
1 bit	8 bit	23 bit

Precisione Doppia:

Rappresentazione su 64 bit – 8 byte

segno	Esponente	Mantissa
1 bit	11 bit	52 bit

La rappresentazione di un numero decimale in virgola mobile avviene secondo i seguenti passaggi:

1. Converto il numero in binario
2. Sposto la virgola secondo la notazione seguente $(1, M) * 2^E$
3. Aggiungo bias all'esponente.

Esempio:

Convertire il numero 13,25 in virgola mobile:

1. $13,25_{10} = (1101,01)_2$
2. $1101,01 \rightarrow 1,10101$ sposto la virgola di tre posizioni modo da ottenere il numero nella notazione corretta, il grado dell'esponente sarà $3 = 2^3$
3. Aggiungo bias (3) all'esponente $127 + 3 = 130$

Logica combinatoria:

Circuiti digitali:

Un circuito combinatorio è un determinato circuito in cui lo stato delle uscite dipende solo dalla funzione logica applicata allo stato istantaneo (cioè in un determinato istante di tempo) delle sue entrate.

Nell'elettronica digitale gli ingressi e le uscite possono assumere solo due tipi di valori, segnale alto corrispondente a 1 e segnale basso corrispondente a 0.

Le porte logiche sono i componenti che permettono di svolgere le operazioni logiche primitive, generalmente prendono in input n segnali e restituiscono un unico output effettuando un'operazione booleana sugli ingressi.

Si distinguono in:

- Porte logiche fondamentali: AND, OR, NOT
- Porte logiche derivate: NAND, NOR, XOR

Porta logica AND:

Svolge l'operazione logica di AND tra due bit, detta anche prodotto logico.

A	B	A*B
0	0	0
0	1	0
1	0	0
1	1	1



Porta logica OR:

Svolge l'operazione logica di OR tra due bit, detta anche somma logica.

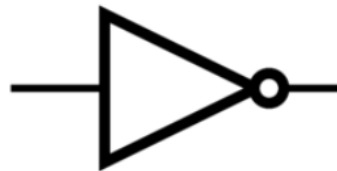
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1



Porta logica NOT:

La porta NOT svolge l'operazione di negazione logica su un bit.

A	$\bar{A}$
0	1
1	0



Porte logiche derivate:

Le porte logiche derivate sono realizzate a partire da quelle fondamentali e permettono una semplificazione dei circuiti.

Porta logica NAND:

La porta logica NAND svolge l'operazione logica di NOT su bit risultante dall'operazione AND di un bit in ingresso.

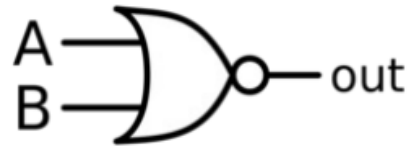
A	B	A*B	A NAND B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0



Porta logica NOR:

La porta logica NOR svolge l'operazione logica di NOT su bit risultante dall'operazione OR di un bit in ingresso.

A	B	A+B	A NOR B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0



Porta logica XOR:

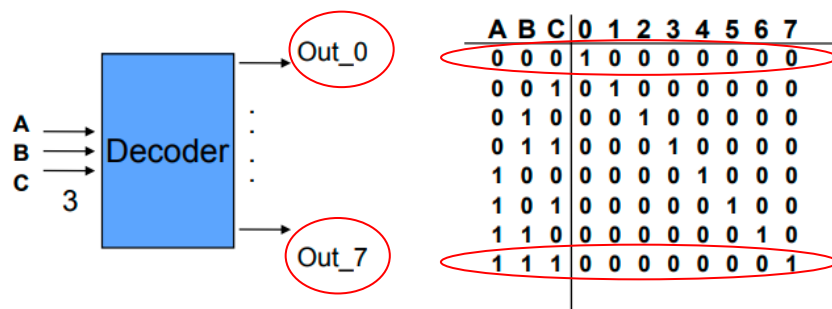
La porta XOR opera come disgiunzione esclusiva tra due input.

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1



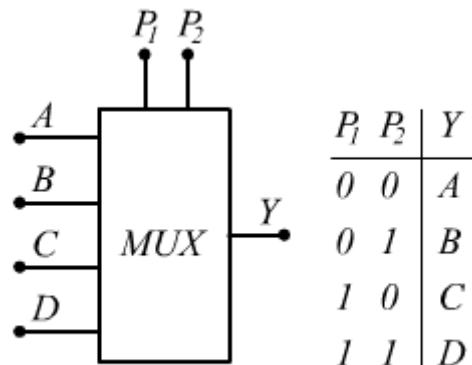
Decoder:

Il decoder è un componente elettronico munito di n ingressi e 2^n uscite, ha lo scopo di impostare allo stato alto l'uscita corrispondente alla conversione in base 10 della codifica binaria a n bit ricevuta in input, lasciando allo stato basso tutte le altre.



Multiplexer:

Il multiplexer è un circuito capace di selezionare uno tra i vari ingressi possibili e di trasferire il dato in esso presente in uscita. È sempre dotato di uno o più ingressi di selezione: m ingressi di selezione servono per pilotare $n = 2^m$ ingressi dati.



Dunque, tra gli n ingressi dati e gli m ingressi di selezione esiste la relazione:

$$n = 2^m \leftrightarrow m = \log_2 n$$

ALU (arithmetic logic unit):

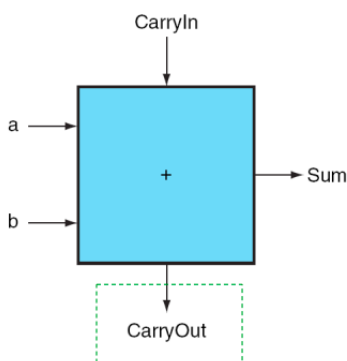
L'alu è la parte del processore che svolge operazioni aritmetico-logiche, e costituita da:

- And gate
- Or gate
- Inverter (not)
- Multiplexer

Esempio di ALU ad 1 bit:

Operazione di somma:

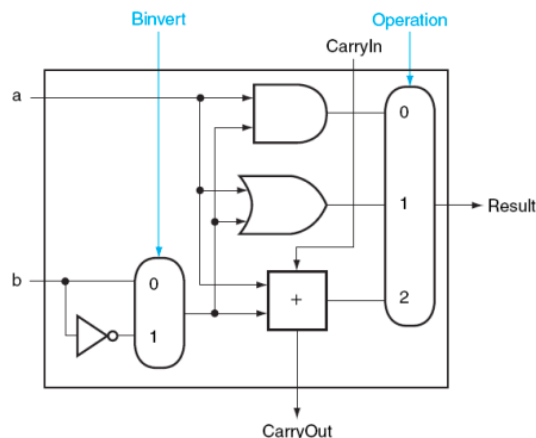
Prende in input tre valori, due numeri con cui effettuare la somma e un terzo valore, il carryIn, ovvero l'eventuale riporto dell'operazione precedente. Restituisce in output la somma dei valori e un eventuale carryOut.



Operazione di sottrazione:

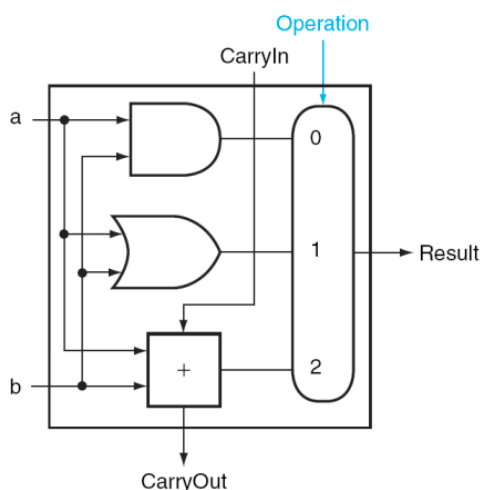
Per implementare la sottrazione in (CA2) è necessario negare l'input b e settare in carryIn fisso a 1. Secondo la seguente formula:

$$a - b = a + (\bar{b} + 1)$$



Addizione And e Or:

L'addizione logica è implementata attraverso l'uso di un multiplexer, con il parametro operation a (2bit) per selezionare l'operazione da eseguire.



Circuiti sequenziali:

I circuiti sequenziali a differenza di quelli combinatori sono in grado di calcolare funzioni che dipendono anche da uno stato, ovvero da informazioni memorizzate da altri elementi di memoria interni.

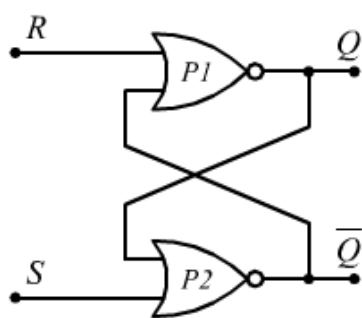
Esistono 2 tipi di circuiti sequenziali:

1. Asincroni, non fanno uso di clock (SR-latch)
2. Sincroni, fanno uso di clock (Flip-Flop)

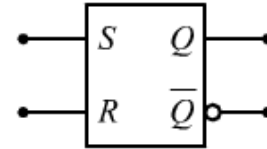
Sr-latch:

è un circuito costituito da due porte NOR concatenate:

- S = set (impostare)
- R = Reset (azzerare)
- Q = terminale di uscita
- $\bar{Q}$ = complementare del terminale di uscita.



S	R	Q_{n+1}	$\bar{Q}_{n+1}$	
0	0	Q_n	$\bar{Q}_n$	memoria
0	1	0	1	reset
1	0	1	0	set
1	1	X	X	non ammesso



Se $S = R = 0$, le porte NOR non dipendono dagli input bensì dai valori di Q e $\bar{Q}$, tali porte si comportano come degli invertitori, ovvero invertono il segnale nel suo complementare a seconda che l'input sia 0 o 1. La condizione $S = R = 0$ è una condizione di mantenimento delle uscite, ovvero di memorizzazione dei valori logici precedentemente assunti.

Se $S = 0$ ed $R = 1$, la porta P1 sta ricevendo in input 1, commuta la sua uscita; la porta P2, a sua volta, avendo in ingresso $S = 0$ e $Q = 0$ assume in uscita il livello logico $\bar{Q} = 1$.

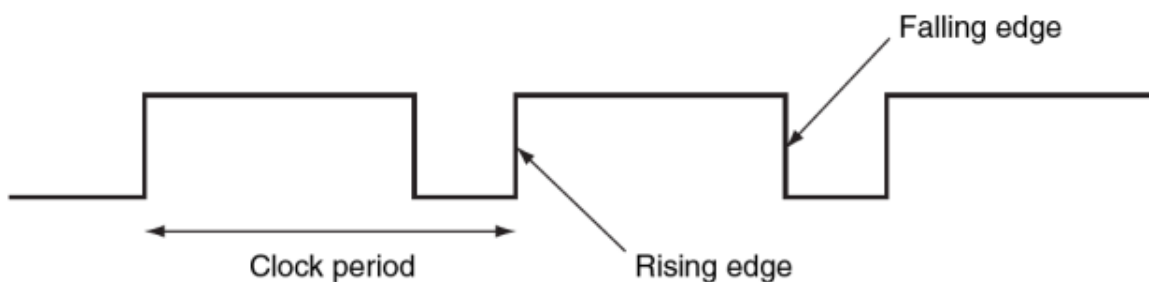
Se $S = 1$ ed $R = 0$ L'uscita della porta P2, che riceve in ingresso un 1, diventa o rimane $\bar{Q} = 0$, di conseguenza l'uscita di P1, avendo entrambi gli ingressi a 0 commuta a livello logico 1. La condizione $S = 1$ ed $R = 0$ che ha come ingresso attivo S (set=imposta) mentre R è inattivo, provoca l'impostazione dello stato logico 1 sull'uscita Q.

La configurazione $S = 1$ ed $R = 1$ viola la proprietà di complementarità di Q e NotQ e può portare ad una configurazione instabile.

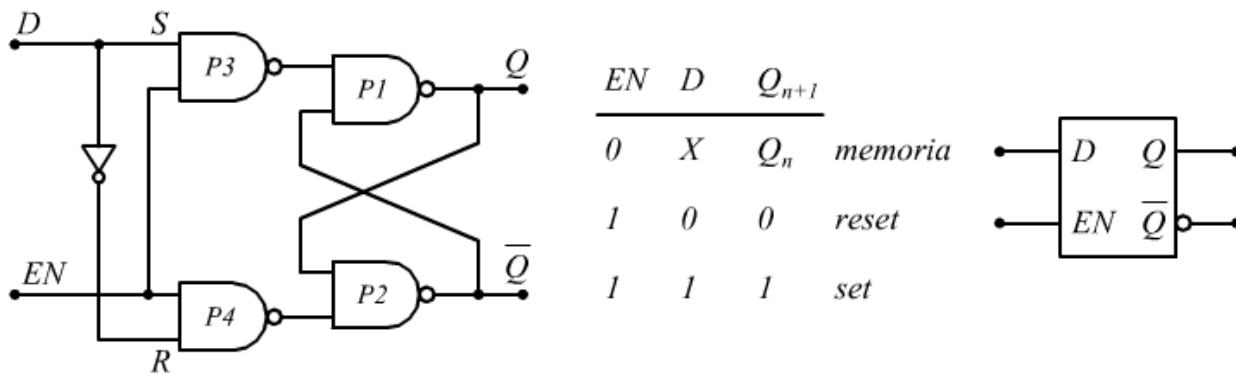
Clock:

Il clock è utilizzato per abilitare la scrittura nei latch, determina il ritmo dei calcoli e delle relative operazioni di memorizzazione.

Il segnale di clock è definito come un'onda quadra con un periodo predeterminato e costante. È caratterizzato da un periodo T e una frequenza F pari a $1/T$ misurata in Hz.



Latch di tipo D:



Il latch di tipo D utilizza un singolo ingresso dati. Ammette solo due configurazioni che corrispondono a $S = 0$ ed $R = 1$ e $S = 1$ ed $R = 0$.

Se $EN=1$ la prima delle due configurazioni ($D=0$) pone il latch in uno stato di reset ($Q=0$) mentre la seconda ($D=1$) lo pone in stato di set ($Q=1$). Pertanto, se $EN=1$ il latch è trasparente rispetto al dato posto su D e l'uscita Q segue l'ingresso D. Se invece $EN=0$ le porte di controllo sono disabilitate e il latch conserva il dato memorizzato in precedenza, prima della disabilitazione.

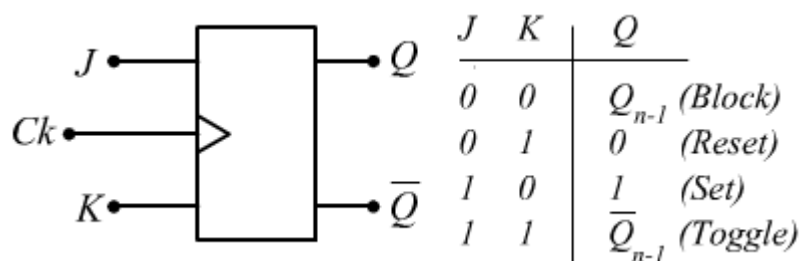
Metodologia di timing Edge-triggered:

Avviene sul fronte di salita o di discesa del clock e premette una memorizzazione istantanea, scongiurando un possibile ritorno di segnale.

Flip Flop:

È la forma più comune di memoria in grado di memorizzare un bit. Possiede due stati stabili che si mantengono nel tempo se non intervengono cause esterne a modificarli.

Il D flip flop ha un unico segnale in entrata, quando il segnale si trova sul fronte di salita di clock, ovvero a 1, il flip flop aggiorna lo stato in Q, nei casi restanti il flip flop conserva lo stato logico corrente indipendentemente dallo stato di D.



Automi a stati finiti:

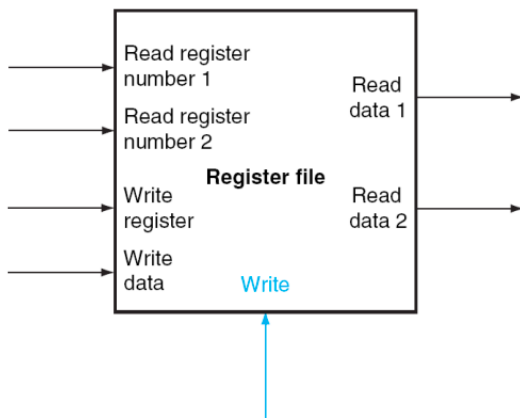
Sono dei sistemi sequenziali, caratterizzati da un numero finito di variabili d'ingresso, d'uscita e di stato. Essi evolvono attraverso una successione di stati dovuti alle sollecitazioni ricevute e forniscono risposte che dipendono dagli ingressi e dagli stati precedenti.

Esistono due modelli per la rappresentazione:

1. Modello Moore: le uscite sono funzione dello stato raggiunto.
2. Modello Mealy: le uscite sono funzione dello stato raggiunto e degli ingressi.

Registri:

I registri sono strutture elaborate in grado di immagazzinare più bit di informazione. Possono essere costruiti attraverso varie configurazioni circuitali con un prefissato numero n di flip-flop, con il terminale di clock e di reset comuni, in questo modo sono in grado di memorizzare n bit.



La struttura intorno alla quale è costruita l'unità di elaborazione è il register file. È formato da un insieme di registri che possono essere letti o scritti fornendo il numero del registro a cui accedere. Un register file può essere realizzato mediante un decodificatore per ciascuna porta di lettura o scrittura ed una matrice di registri costruiti a partire da flip flop di tipo D. Poiché la lettura di un registro non ne modifica lo stato, occorre solo fornire in ingresso il numero del registro ed in uscita si otterrà semplicemente il dato contenuto nel registro. Per la scrittura in un registro sono invece richiesti tre ingressi: il numero di registro, il dato da scrivere ed un segnale di clock che controlli l'operazione di scrittura. Le porte in lettura si possono implementare con due multiplexer, ciascuno dei quali di ampiezza pari al numero di bit nel register, il file letto sarà il valore scritto in un periodo di clock precedente. Se si volesse leggere il valore attualmente in corso di scrittura, sarebbe necessaria della logica aggiuntiva nel register file o al suo esterno.

Gerarchia delle memorie:

Idealmente un calcolatore dovrebbe avere quanta più memoria possibile, ad alta velocità, basso consumo e minimo costo; tuttavia, ciò non è possibile pertanto si possono organizzare i vari tipi di memoria in una sorta di gerarchia. Le memorie piccole, più veloci e costose sono poste ai livelli alti, ovvero più vicine al processore, mentre le memorie più ampie, più lente e meno costose sono poste ai livelli più bassi quindi più lontano dal processore.

Memoria RAM (random access memory):

La dimensione del Register File è piccola, per memorizzare dati strutturati e codice di programma, abbiamo bisogno di memorie più grandi come ad esempio la memoria RAM, meno veloce della memoria dei registri ma molto più capiente. La memoria ram è costituita da un insieme di celle di grandezza pari a un byte ognuna individuata da un indirizzo. La memoria ram è in grado di compiere due operazioni la lettura e la scrittura, tramite l'indirizzamento seleziona una determinata cella di memoria. Con n bit si indirizzano 2^n celle di memoria. Nella fase di lettura il contenuto della cella di memoria indirizzata dal Registro Indirizzi è copiato nel Registro Dati, nella fase di scrittura il contenuto del Registro Dati è copiato nella cella di memoria indirizzata dal Registro Indirizzi.

SRAM:

L'SRAM (Static Random Access Memory) è un circuito integrato che funziona come un array di memoria con un unico input per operazioni di read/write. La singola cella è composta da un array di transistor che mantengono il dato in memoria in modo indefinito richiedendo una potenza minima e non necessitando di refreshing come la DRAM. L'SRAM ha un tempo di accesso uguale per ogni dato anche se le performance di read/write possono differire.

DRAM:

A differenza dell'SRAM nella DRAM (Dynamic Random Access Memory) l'unità di memoria di base è un condensatore e richiede un solo transistor per leggere o scrivere. Questa caratteristica la rende molto più compatta, riduce il costo per bit ma allo stesso tempo risulta più lenta. Dal momento che utilizza un condensatore (che perde la carica in un paio di millisecondi) il dato necessita di essere riscritto costantemente (semplicemente si legge il dato e lo si riscrive). Se si dovessero refreshare i bit uno ad uno non ci sarebbe il tempo per eseguire altre operazioni, per ovviare a questo problema si utilizza un decoder a due stadi che permette di refreshare un'intera riga allo stesso momento. Un ulteriore miglioramento nelle performance è dato dall'uso di un buffer che funziona come un'SRAM: i dati della riga richiesta vengono salvati nel buffer e sono accessibili senza necessità di refresh finché non si richiede la lettura di un'altra riga.

ISA (instruction set architecture):

Filosofie di progetto dei processori:

Esistono due metodologie per la progettazione dei processori:

1. CISC: Utilizza più hardware, muove dati da memoria a memoria
2. RISC: Utilizza più software e trasferisce i dati tra i registri.

Architettura MIPS:

MIPS appartiene alla categoria RISC, per rappresentare le informazioni utilizza 32 registri da 32 bit ciascuno. È in grado di manipolare dati solamente sui registri, trasferire dati tra memoria e registri e compiere delle alterazioni del flusso di controllo eseguendo dei salti tra i registri. Queste operazioni vengono realizzate seguendo tre tipi di istruzioni principali:

- Istruzione R-Type.
- Istruzione I-Type.
- Istruzione J-Type.

Istruzione R-Type:

L'istruzione R-Type è costituita da un OP (operation code) che identifica il tipo di operazione che voglio svolgere, un primo registro sorgente, un secondo registro sorgente, il registro destinazione (Rd), lo shift che generalmente è settato a 0 e da una function che mi permette di capire la variante dell'operazione.

Una possibile istruzione è la seguente: "somma i contenuti del registro 8 e del registro 9 e metti il risultato nel registro 10".

Le istruzioni R-Type sono di tipo: add, sub, mul, div, and, or.

000000	01000	01001	01010	00000	10000
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit
OP	rs	rt	rd	Shamt	Funct
(Operation code) tipo di operazione	Primo registro sorgente	Secondo registro sorgente	Registro destinazione	SHAMT (shift amount)	FUNCT variante dell'operazione

Esempio:

Operazione add: l'operazione di addizione salva nel registro destinazione (rd) la somma dei valori contenuti nei registri rs e rt.

Istruzione I-Type (addition immediate):

001000	01000	01010	0000000000000100
6 bit	5 bit	5 bit	16 bit
OP	rs	rt	Valore immediato
Operation code	Primo registro sorgente	Registro destinazione	Costante di indirizzo

Operazione ADDI: l'operazione addition immediate salva nel registro di destinazione (rt) la somma del registro sorgente (rs) del valore immediate.

001000	01000	01010	0000000000000100
6 bit	5 bit	5 bit	16 bit
Operation code	Primo registro sorgente	Registro destinazione	Valore immediato

Operazione LW (load from memory): l'operazione load word carica la parola di 32 bit dalla memoria all'indirizzo di destinazione rt.

Operazione SW (store from memory): l'operazione store prende i dati dal registro destinazione (memoria) per caricarli all'interno di un altro registro.

Operazione Branch: (branch equal to (BEQ) -> equivalente all'istruzione if-else). L'istruzione compara i valori nel primo e nel secondo registro e se sono uguali continua sul registro rappresentato dall'else.

001000	01000	01010	0000000000000100
6 bit	5 bit	5 bit	16 bit
OP	rs	rt	Branch address
Operation code	Primo registro sorgente	Registro destinazione	Branch address

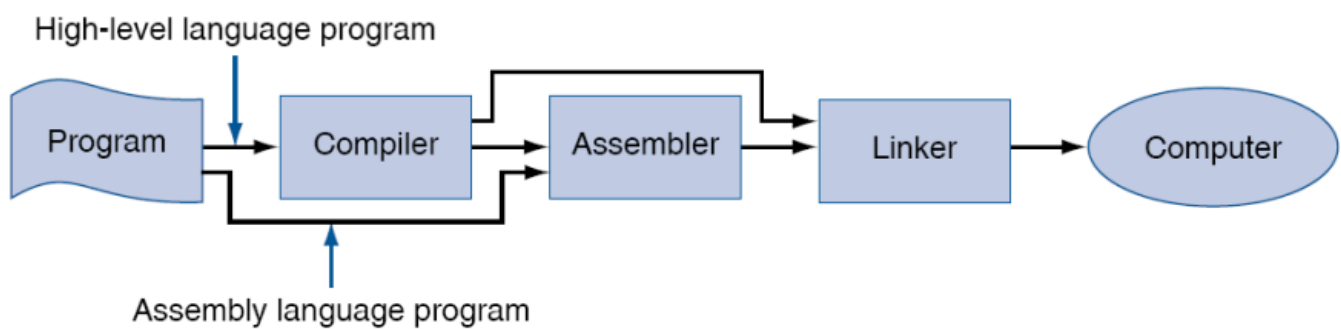
Istruzione J-Type:

00010	0000 0000 0000 0001 0000 1110 10
6 bit	26 bit
Operation Code	Jump word address

L'istruzione Jump trasla l'indirizzo alla word più vicina, l'indirizzo a disposizione è di 26 bit. In MIPS gli indirizzi hanno una lunghezza pari a 32 bit, l'indirizzo verrà quindi completato nei suoi 4 bit più significativi dai 4 bit del Program counter, portando la lunghezza a 30 bit. I due bit restanti saranno rappresentati da 2 zeri posti alla fine dell'indirizzo. Essendo tutti gli indirizzi multipli di 4 i bit finali saranno sempre pari a 0.

Programmazione Assembly:

Catena programmatica:



La catena programmatica consiste nel percorso che svolge il codice dalla sua scrittura fino al momento in cui viene eseguito, ed è articolata in più passaggi.

Il programma in linguaggio ad alto livello passa attraverso il compilatore e viene compilato in linguaggio macchina; tuttavia, il programma può essere scritto in assembly e passato direttamente all'Assembler.

Linker:

Il linker (link editor o collegatore) ha il compito di collegare tra loro vari moduli che compongono lo stesso programma. Il programma sorgente è costituito da più file di **object** che vengono associati con funzioni di libreria per creare l'eseguibile **.exe** che può essere utilizzato dall'hardware.

Loader:

Il loader è un elemento del sistema operativo non facente parte della catena programmatica, ha il compito di predisporre l'ambiente di esecuzione e memoria svolgendo i seguenti passaggi:

- Leggere l'intestazione per determinare la dimensione dei segmenti testo e dati
- Crea uno spazio di memoria sufficiente per testo e dati
- Copia le istruzioni e i dati in memoria
- Copia nello stack i parametri (se presenti) passati al main.
- Inizializza i registri e lo stack pointer.

- Salta ad una procedura di inizializzazione che copia i parametri nei registri appostiti dallo stack e poi invoca la procedura di inizio del programma

Assembler:

L'assemblatore ha il compito di svolgere i seguenti passaggi:

1. Converte un programma assembly (file sorgente) in linguaggio macchina (file oggetto).
2. Gestisce le etichette.
3. Traduce i codici mnemonici e riferimenti simbolici in binario.
4. Gestisce pseudo istruzioni.
5. Converte i valori esadecimali/decimali in binario.

Le etichette di salto possono avere dei riferimenti a codice che si trova più avanti nell'istruzione o in altri file; pertanto, l'Assembler effettua due passate per trovare tali indirizzi.

Pseudo-istruzioni:

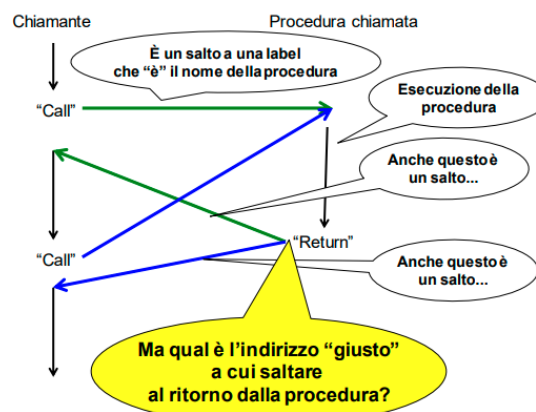
Le pseudo-istruzioni sono istruzioni non implementate dall'hardware cioè, non native dell'ISA di MIPS, vengono quindi tradotte dall'assemblatore in sequenze di istruzioni native. Secondo la convenzione d'uso dei registri, il registro \$1 (\$at) è riservato a questo scopo. Ad esempio, l'istruzione LA -> load address è una pseudo istruzione a 32 bit che viene divisa in due sotto istruzioni di 16 bit ciascuna (load upper immediate e or immediate).

Passaggio di parametri:

Quando viene chiamata una funzione in assembly, l'istruzione si muove attraverso dei blocchi; perciò, mi serve un'istruzione jump che riesca a ricordare l'indirizzo di partenza della funzione. L'istruzione **jump and link (JAL)** mi permette sia di eseguire il chiamato, sia di effettuare il salvataggio dell'indirizzo a cui tornare. L'istruzione di ritorno prevede che venga salvato in **\$ra** (return address) l'indirizzo di ritorno. Con l'istruzione **jump register \$ra** il sistema guarda il numero scritto dentro al registro che corrisponde all'indirizzo a cui saltare.

Complicazioni di jump register:

1. Mi devo ricordare di fare jump register su \$ra perchè gli altri registri non contengono l'indirizzo di ritorno.
2. Quando devo effettuare più chiamate jump and link non funziona perché vengono sovrascritti gli indirizzi, l'unica soluzione è utilizzare un registro di appoggio salvando l'indirizzo che mi interessa.



Registri temporary (\$t):

I registri temporary sono dei registri che non vengono salvati dalla procedura; pertanto, il chiamante non troverà il contenuto di tali registri immutato dopo aver chiamato la procedura, dovrà quindi effettuare un salvataggio prima di una qualsiasi chiamata.

Registri saved (\$s):

I registri saved sono dei registri che rimangono immutati dopo la chiamata di una procedura. Se la procedura utilizza registri saved deve salvarne il contenuto e ripristinarlo prima del ritorno.

Registri argomento(\$a):

Il passaggio degli argomenti ad una funzione chiamata avviene tramite i registri \$a0.

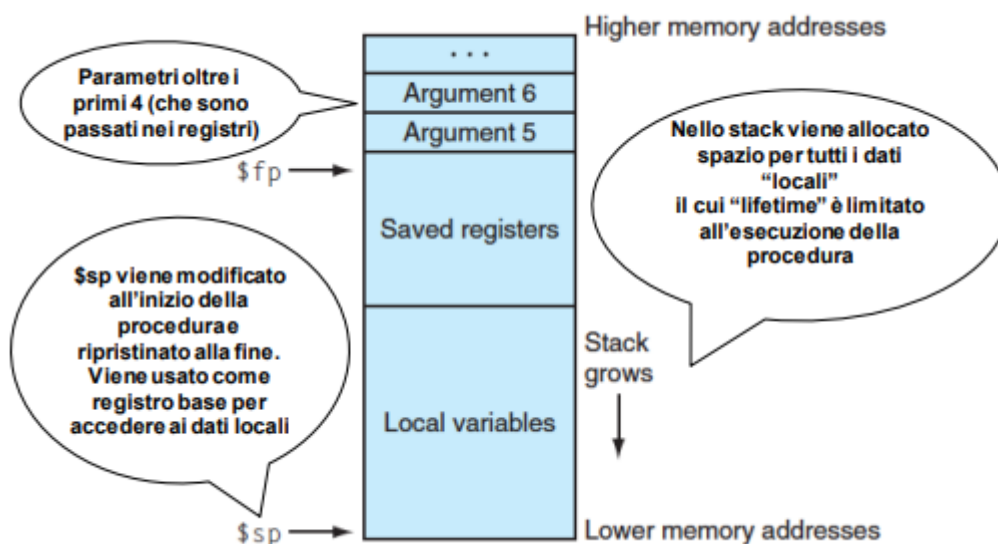
Registri valore(\$v):

I registri valore \$v vengono utilizzati per registrare il valore di ritorno che restituisce una funzione, il chiamato viene registrato nei registri \$v.

Stack:

Lo stack segment è la zona di memoria dove vengono impilati i vari salvataggi a scendere, quando eseguo un ripristino lo stack viene svuotato. Il punto di inizio dello stack è un punto fisso, la fine invece dipende dal numero di salvataggi che ho in memoria, lo stack ha una gestione di tipo **LIFO (last in – first out)**, gli ultimi salvataggi sono i primi ad essere eliminati.

All'interno dello stack ci sono due tipi di puntatori, il puntatore **\$sp (stack pointer)** che indica il registro da cui posso andare ad utilizzare i dati salvati e il puntatore **\$fp (frame pointer)** indica la fine della memoria. Quando la procedura deve salvare alloca lo spazio e sposta lo stack pointer, quando riprende aggiorna lo stack pointer.



Convenzione in assembly nomi e usi dei registri:

Nome Simbolico	Numero	Uso
\$Zero	0	Costante 0
\$at	1	Assembler temporary
\$v0-\$v1	2-3	Functions and expression evaluation
\$a0-\$a3	4-7	Arguments
\$t0-\$t7	8-15	Temporaries
\$s0-\$s7	16-23	Saved Temporaries
\$t8-\$t9	24-25	Temporaries
\$k0-\$k1	26-27	Reserved for OS kernel
\$gp	28	Global pointer
\$sp	29	Stack pointer
\$fp	30	Frame pointer
\$ra	31	Return address

Datapath:

Datapath singolo ciclo:

Nel funzionamento a ciclo singolo, il ciclo è di lunghezza fissa e corrisponde all'istruzione più lunga che deve essere eseguita, ogni istruzione viene eseguita in un ciclo di clock. I componenti vengono utilizzati una sola volta per ciascuna istruzione, di conseguenza eseguire operazioni più complesse richiede l'utilizzo di più componenti.

Componenti del Datapath:

1. Program counter:

Usato per memorizzare l'indirizzo dell'istruzione corrente, dopo il prelevamento dell'indirizzo necessario nella fase di fetch dell'istruzione il PC viene incrementato di 4 per puntare la prossima istruzione.

2. Register file:

Una collezione di registri in cui è possibile leggere o scrivere ogni registro specificando il numero del registro interessato. Il register file contiene inoltre il registro di stato della macchina. Gli ingressi per la selezione dei registri hanno ampiezza di 5 bit per consentire l'indirizzamento di $2^5 = 32$ registri, mentre i bus relativi ai dati hanno ampiezza di 32 bit.

3. ALU:

L'ALU possiede 2 ingressi da 32 bit ciascuno e produce un output da 32 bit, e un output di 1 bit se il risultato è 0, è dotata inoltre di un bus da 4 bit per il controllo delle sue funzioni (AluOp).

4. Sign-Extend: Ripete il bit più significativo dei 16 bit di offset fino ad ottenere 32 bit.

5. ALU-Control:

Possiamo generare un input di 4 bit usando una piccola control-unit che ha come input il campo function dell'istruzione e 2 bit di controllo denominati ALUOp, l'ALUOp indica quale operazione deve effettuare l'ALU:

- 00 -> esegue una somma.

- 01 -> esegue una sottrazione.
- 10 -> esegue quello che è indicato dal campo funzione.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

Passi per l'esecuzione di una istruzione:

- **FETCH:** è una procedura comune a tutte le istruzioni, legge l'istruzione dalla memoria e la salva in un registro dedicato, l'Instruction Register, l'indirizzo di memoria che indica l'istruzione da leggere si trova nel registro PC. Dopo la lettura il PC viene incrementato di quattro per indicare la prossima istruzione da leggere.
- **DECODE:** decodifica i vari campi dell'istruzione per decidere quali sono i passi necessari per la sua esecuzione.
- **EXECUTE:** esegue i passi necessari per eseguire l'istruzione.

Le prime due fasi sono comuni a tutte le istruzioni e durano un solo colpo di clock, la fase di execute varia in base al tipo di istruzione così come la sua durata.

Quali operazioni esegue un processore:

Il processore per ogni istruzione:

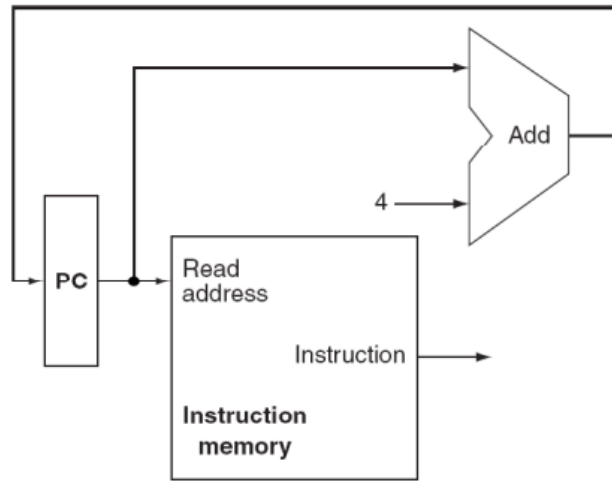
- legge dalla memoria l'istruzione il cui indirizzo è nel PC.
- legge dei registri (in base a quanto specificato dall'istruzione).

A seconda dell'istruzione:

- Usa la ALU per calcolare uno tra i seguenti valori:
 - a. un risultato aritmetico/logico
 - b. un indirizzo di memoria (registro + offset)
 - c. l'indirizzo di un branch target
- accede alla memoria per load/store.
- aggiorna il PC con $PC + 4$ o con un indirizzo target.

Implementazione della fase di Fetch:

Per eseguire un'istruzione è necessario prima leggerla dalla memoria, Il PC è un registro dedicato che contiene l'indirizzo dell'istruzione attualmente in esecuzione, ogni istruzione è rappresentata da un blocco di 32 bit quindi l'istruzione successiva a quella corrente si troverà all'indirizzo $PC + 4$.



Fase di Decode:

Il processore MIPS legge i vari campi dell'istruzione e identifica il tipo di istruzione da eseguire (OP CODE e FUNC CODE se necessario).

Execute R-Type:

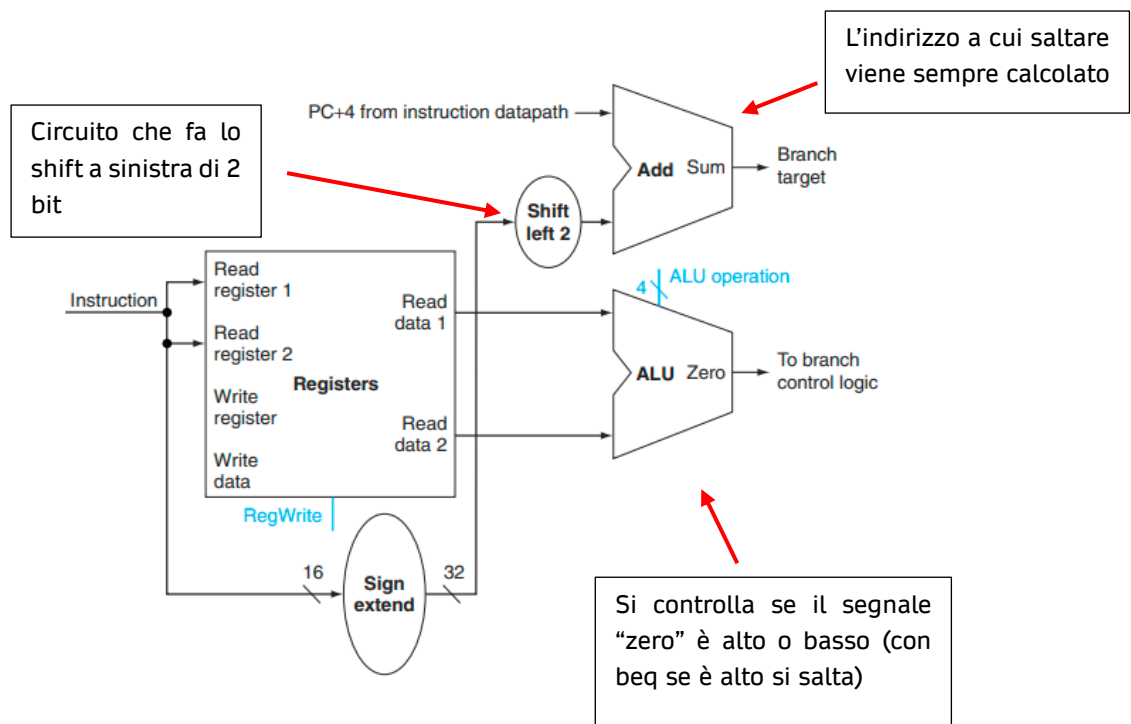
L'istruzione viene letta dalla memoria, vengono selezionati i due registri in lettura (rs e rt) e il registro in scrittura (rd). Vengono generati segnali di controllo dell'ALU, letti i valori dei registri sorgenti e dati in input alla ALU che esegue l'operazione, Il valore in output viene scritto nel registro destinazione.

Execute Beq:

Per le istruzioni di branch abbiamo bisogno di calcolare il branch target address relativo all'istruzione, per fare ciò sommiamo il campo offset esteso di segno con il PC prestando attenzione a 2 particolari:

1. L'indirizzo base per la branch è l'indirizzo dell'istruzione successiva alla branch; quindi, il PC è $PC+4$
2. Deve esserci un allineamento alla word, quindi dopo aver esteso il segno bisogna shiftare di 2 bit a sinistra l'indirizzo prima di sommarlo al PC.

Dunque, il datapath deve eseguire due operazioni, calcolare il branch target address e comparare il contenuto dei due registri, se il risultato dell'uscita zero dell'ALU è alto, allora la Branch ha dato esito positivo



Execute jump:

Per l'istruzione di salto il target address è calcolato come segue:

1. Prendo i 26 bit del campo indirizzo dell'istruzione.
2. Li shifto di 2 bit verso sinistra.
3. Li concateno ai 4 bit più significativi del PC.

Implementazione Multi-ciclo:

In una implementazione multi-ciclo ogni step di esecuzione impiega un ciclo di clock, e permette alle unità funzionali di essere usate per più di una istruzione, questa condivisione aiuta a ridurre l'ammontare di richiesta di hardware. La capacità di consentire alle istruzioni di usare un numero differente di cicli di clock e la abilità di condividere parti funzionali durante l'esecuzione di una singola istruzione sono i maggiori vantaggi dell'implementazione multi-ciclo. Al termine di ogni singolo ciclo tutti i dati usati nei cicli di clock successivi vanno memorizzati, nella nostra implementazione assumiamo che in un ciclo di clock possono rientrare al massimo una di queste operazioni:

- Un accesso alla memoria.
- Un accesso al register file.
- Un'operazione dell'ALU.

Alla fine del ciclo ogni output prodotto da una di queste operazioni deve essere salvato in un registro temporaneo.

Registri temporanei:

- Instruction Register (IR) e il Memory data register (MDR) per salvare rispettivamente l'istruzione letta dalla memoria e la lettura di un dato, mantiene l'istruzione in memoria fino alla fine dell'esecuzione dell'istruzione.
- I registri A e B sono usati per mantenere il valore degli operandi letti dal register file.
- Il registro ALUOut salva il risultato dell'ALU.

Per le istruzioni di branch e salto ci sono 3 possibili sorgenti del valore da scrivere nel PC:

1. L'uscita dell'ALU che è il PC+4, questo valore va memorizzato direttamente nel PC.
2. Il registro ALUOut dove memorizzeremo l'indirizzo calcolato per la branch.
3. I 26 bit meno significativi del IR shiftati di 2 verso sinistra e concatenati con i 4 bit più significativi del PC+4, questa è la sorgente per le istruzioni di jump.

Useremo quindi due controlli per modificare il valore del PC, PCWrite che prevede una scrittura incondizionata del PC, e PCWr_Cond che esegue una scrittura del PC solo nel caso la condizione risulti vera.

Eccezioni:

Le eccezioni sono degli eventi inattesi che si possono verificare durante l'esecuzione di un programma, esistono due tipi di eccezioni:

1. **Eccezioni sincrone:** avvengono in concomitanza di un'istruzione che viene svolta, in particolare durante le tre fasi di esecuzione dell'istruzione (fetch-decode-execute). La condizione di eccezione deve essere risolta da un gestore di eccezioni, se è risolubile, allora il programma può riprendere l'esecuzione; altrimenti, il programma termina prima della sua fine.
2. **Eccezioni asincrone (interruzioni):** sono delle eccezioni che si verificano durante l'esecuzione del programma per cause esterne allo stesso, non avvengono all'interno del processore. Sono gestite tra due istruzioni consecutive, si sospende l'esecuzione del programma utente, si gestisce l'interruzione e poi si riprende l'esecuzione del programma utente.

Registro cause:

Il registro cause viene utilizzato per memorizzare il motivo dell'eccezione.

Cause exceptions:

Number	Name	Cause of exception
0	Int	interrupt (hardware)
4	AdEL	address error exception (load or instruction fetch)
5	AdES	address error exception (store)
6	IBE	bus error on instruction fetch
7	DBE	bus error on data load or store
8	Sys	syscall exception
9	Bp	breakpoint exception
10	RI	reserved instruction exception
11	CpU	coprocessor unimplemented
12	Ov	arithmetic overflow exception
13	Tr	trap
15	FPE	floating point

Registro EPC:

Il registro EPC (exception program counter), formato da 32 bit, viene utilizzato per salvare l'indirizzo dell'istruzione corrente che ha causato l'eccezione.

Gestore delle eccezioni:

I due tipi di eccezione generabili dall'implementazione sono l'esecuzione di **un'istruzione non valida** e **l'overflow aritmetico**. L'operazione fondamentale che il processore deve compiere al verificarsi di un'eccezione consiste nel salvare l'indirizzo dell'istruzione responsabile dell'eccezione nell'Exception Program Counter (EPC) e trasferire il controllo al sistema operativo, ad un qualche indirizzo specifico.

Il sistema operativo sceglierà quindi la soluzione migliore, che potrebbe consistere nel fornire un servizio al programma utente, eseguire un'azione predeterminata in risposta all'overflow o terminare l'esecuzione del programma segnalando un errore.

Dopo aver compiuto le azioni necessarie in risposta all'eccezione, il sistema operativo può terminare il programma o riprenderne l'esecuzione utilizzando EPC per determinare il punto da cui ripartire. Per poter gestire correttamente l'eccezione, il sistema operativo deve conoscerne il motivo, oltre all'istruzione che l'ha causata.

Vi sono due metodi per rendere disponibile il motivo di un'eccezione:

Indirizzo fisso:

il metodo utilizzato nell'architettura MIPS è di includere un registro di causa (detto registro Cause) contenente un campo che indica il motivo dell'eccezione.

Interruzioni vettorizzate:

Un metodo alternativo prevede di adottare le interruzioni vettorizzate: nel corso di un'interruzione vettorizzata, l'indirizzo a cui trasferire il controllo è determinato dalla causa dell'eccezione. Ad esempio, per gestire i due tipi di eccezione sopra elencati, si potrebbero definire i seguenti indirizzi.

Tipi di eccezione	Indirizzo del vettore di eccezione (esa)
Istruzione indefinita	C0000000 ₁₆
Overflow aritmetico	C0000020 ₁₆

Il sistema operativo ricava il motivo dell'eccezione dall'indirizzo a cui viene attivato; gli indirizzi sono separati da 32 byte, equivalenti ad 8 istruzioni, ed in questa breve sequenza il sistema operativo deve registrare il motivo dell'eccezione ed eseguire qualche semplice elaborazione. Quando l'eccezione non è vettorizzata, tutte le eccezioni utilizzano lo stesso punto iniziale ed il sistema operativo deve decodificare il registro di stato per conoscere la causa.

Si assuma di voler implementare il sistema di eccezioni utilizzato nell'architettura MIPS, occorrerà aggiungere due registri all'unità di elaborazione, saranno necessari due segnali di controllo aggiuntivi per la scrittura dei registri EPC e Cause, detti **EPCWrite** e **CauseWrite**, inoltre servirà un segnale di controllo aggiuntivo **PCsrc** di 1bit per fornire il valore corretto al bit meno significativo del registro Cause, chiamato **CauseInt**. Infine, occorrerà poter scrivere nel registro PC l'indirizzo iniziale della procedura di gestione delle eccezioni, per esempio C0000000|₁₆.

Attualmente PC è alimentato dall'uscita di un multiplexer a tre vie, controllato dal segnale PCSource: lo si può trasformare in un multiplexer a quattro vie, con l'ingresso aggiuntivo collegato

al valore costante $C0000000|_{16}$ ed un valore di PCSource pari 11_2 selezionerà tale valore. Siccome il PC viene incrementato nel primo ciclo, per poter salvarne il valore nell'EPC è necessario utilizzare l'ALU e sottrarre 4 al valore del PC, l'input dell'EPC è collegato all'output dell'ALU. Nel momento in cui si verifica un'eccezione si salvano i dati nei registri per non mutarli durante la gestione dell'eccezione, in caso di eccezioni annidate si procede con la FIFO (first in first out).

I/O memory mapped:

Per la gestione delle periferiche si usa la parte di memoria oltre la locazione $0x7FFFFFFC$, normalmente riservata al sistema. La periferica si interfaccia verso l'esterno con i registri periferica, in particolare con il registro di stato e il registro dati. Il registro di stato rappresenta lo stato della periferica che viene letto dalla CPU, lo stato contiene informazioni riguardanti la prontezza della periferica per una trasmissione di dati o la disponibilità dei dati richiesti per il trasferimento. Il registro dati è di input o output per la CPU a seconda del tipo di periferica.

Ci sono diversi modi in cui la CPU può controllare lo stato della periferica, il problema si verifica quando la CPU esegue istruzioni con una determinata frequenza che non viene pareggiata dalla periferica nella generazione dei dati.

Valutazione della prestazione:

Banda passante: rappresenta la quantità di dati che si può trasferire per unità di tempo (più è alta meglio è) byte/s unità di misura di flusso -> è una quantità fisica.

Latenza: rappresenta il tempo che intercorre tra il momento in cui la periferica è pronta e il tempo in cui viene trasferito il dato, indica quanto è veloce a rispondere il sistema.

Controllo di programma:

La CPU controlla il valore del registro di stato della periferica finché questa non risulta pronta, esegue questo controllo continuamente e ciò comporta uno spreco sia di energia che di tempo; infatti, non permette alla CPU di eseguire altre istruzioni. Il tempo d'attesa utilizzato per eseguire le istruzioni del ciclo prende il nome di Busy Wait. Nel Momento in cui la periferica diventerà pronta la CPU eseguirà immediatamente il trasferimento. Il trasferimento si ottiene caricando prima il valore nei registri della CPU e poi con una lw/sw in memoria.

Latenza: minima in quanto la CPU noterà subito lo stato della periferica.

Banda passante: alta perchè la CPU trasferirà subito il dato e la gestione della periferica richiede poche istruzioni. dipende anche dai colli di bottiglia (se c'è qualsiasi intoppo la banda si abbassa in quanto le istruzioni sono più lente).

I/O guidato da interrupt:

Per eliminare l'attesa del processore utilizziamo l'approccio con l'interrupt, il processore dopo aver avviato la prima richiesta di dati può svolgere altre operazioni, quando la periferica è pronta ad essere utilizzata manda un interrupt al processore. Un vantaggio dell'utilizzo di un interrupt è il fatto che la CPU non fa busy waiting, d'altra parte la CPU deve comunque gestire le operazioni di trasferimento.

Latenza: maggior latenza per la maggior quantità di operazioni da eseguire.

Banda passante: minor banda passante in quanto il trasferimento di ogni dato necessita di più tempo.

DMA – accesso diretto alla memoria

Questo metodo viene utilizzato quando è necessario spostare una grande quantità di dati, la periferica diventa autonoma con gli accessi in memoria e gestisce i trasferimenti senza l'aiuto del processore. Necessita di due registri aggiuntivi oltre al registro di stato e al registro dati, i due registri in questione sono un registro che indica l'indirizzo di memoria da dove trasferire i dati e un registro che segna la quantità di dati da trasferire. Alla fine del trasferimento la periferica invia un interrupt alla CPU per segnalare il completamento del trasferimento.

Latenza: minima dato che nessuna istruzione è eseguita dalla CPU.

Banda passante: massima perchè la CPU non deve eseguire nessuna istruzione nessuna istruzione.

Controllo del percorso dei dati in pipelining:

Il pipelining è una tecnica di implementazione hardware di un insieme di istruzioni che prevede la sovrapposizione temporale dell'esecuzione delle diverse istruzioni. Tutti gli stadi della pipeline sono attivi contemporaneamente, purché si abbiano risorse separate per ciascuno stadio.

la pipeline aumenta il throughput (cioè il numero delle istruzioni eseguite in un'unità di tempo). Il motivo per cui le pipeline sono più veloci è che tutte le unità lavorano in parallelo; quindi, si possono eseguire più cicli di istruzioni in minor tempo.

L'esecuzione di un'istruzione MIPS in pipeline richiede fino a 5 passaggi:

1. Fetch: caricamento dell'istruzione dalla memoria.
2. Decode: lettura dei registri e codifica delle istruzioni (simultanee).
3. Execute: esecuzione delle istruzioni.
4. Memory: leggere o scrivere un dato in memoria.
5. Write back: salvataggio di un risultato nel registro destinazione.

Hazards:

In una pipeline si possono verificare situazioni in cui l'istruzione successiva non può essere eseguita nel ciclo di clock immediatamente successivo, tali eventi sono detti hazard e possono essere di tre tipi.

Hazard strutturali:

Si verificano quando le risorse hardware presenti non sono in grado di supportare la combinazione di istruzioni che si vorrebbero eseguire nello stesso ciclo di clock.

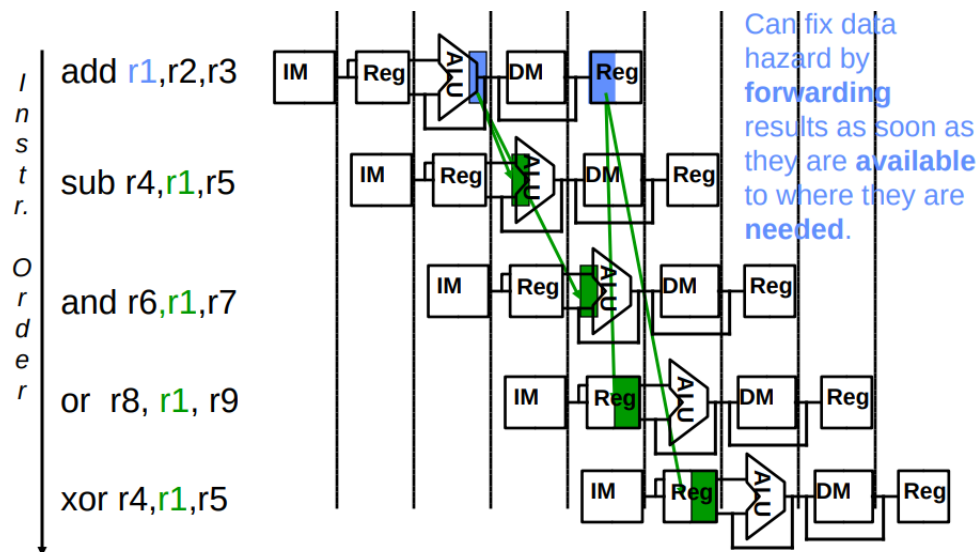
Hazard sui dati:

Si verificano quando la pipeline deve essere messa in stallo in quanto uno stadio deve attendere che termini l'elaborazione di un altro. In un calcolatore dotato di pipeline, gli hazard sui dati accadono quando un'istruzione dipende dal risultato di un'istruzione precedente che si trova ancora all'interno della pipeline. Supponiamo, per esempio, di avere un'istruzione di somma seguita da una sottrazione che utilizza il risultato della somma:

add \$s0, \$t0, \$t1

sub \$t2, \$s0, \$t3

Gli hazards sui dati possono causare numerose situazioni di stallo nella pipeline, pertanto, si ricorre alla tecnica del **bypassing (data forwarding)** come soluzione. Tale tecnica consiste nell'aggiunta di un multiplexer che fornisce il dato mancante prendendolo da una risorsa interna, in particolare dall'ALU. In questo caso, infatti, appena viene generato il risultato della somma da parte dell'ALU, questo viene subito riutilizzato per l'operazione di sottrazione.

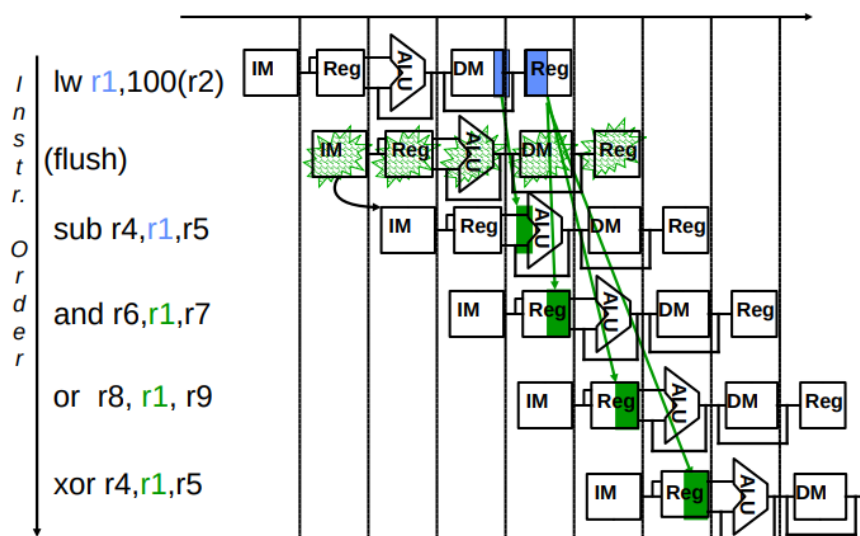


Propagazione nel caso di due o più istruzioni:

Con il bypassing la propagazione del risultato è gestita ottimamente; tuttavia, non si è in grado di evitare gli stalli sulla pipeline.

Supponiamo di avere come prima istruzione la lettura dalla memoria a \$s0 (load), anziché una add, il dato sarebbe disponibile dopo il quarto stadio della prima istruzione, ossia troppo tardi per essere utilizzato come input del terzo stadio della sottrazione. Di conseguenza, anche disponendo di un meccanismo per la propagazione, dovremmo comunque imporre uno stallo della durata di un ciclo di clock a causa di un hazard sui dati di un'operazione di load. Questa situazione prende il nome di stallo della pipeline, spesso viene indicata con il termine "bubble".

Forwarding with Load-use Data Hazards



Hazard sul controllo:

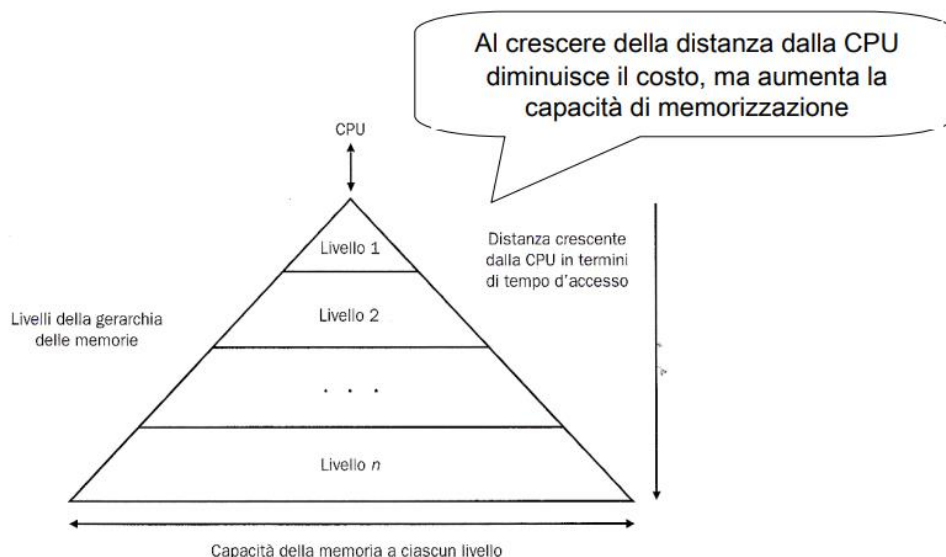
Si verificano quando l'esecuzione di un'istruzione dipende dallo stato e dal completamento di altre istruzioni già avviate.

Supponiamo di dover eseguire un'istruzione di salto condizionato, un'operazione di branch quindi. La lettura dalla memoria istruzione dell'istruzione che segue la branch viene eseguita nel ciclo di clock immediatamente successivo a quello della branch. La pipeline, quindi, non può sapere quale sarà l'istruzione successiva in quanto ha appena ricevuto l'istruzione di branch. Una possibile soluzione al problema prevede che la pipeline venga messa in stallo subito dopo aver caricato dalla memoria un'istruzione di branch e attendere fino al momento in cui il risultato dell'operazione precedente sia disponibile in modo tale da poter calcolare l'istruzione successiva.

Cache:

Gerarchie di memoria:

Una gerarchia di memoria consiste in un insieme di livelli di memoria, ciascuno caratterizzato da una diversa velocità e dimensione.



Memoria cache:

La memoria interna alla CPU è costituita dai registri ed è caratterizzata da alta velocità e limitate dimensioni. La memoria centrale, rispetto a quella interna ha dimensioni molto maggiori che prevedono tempi d'accesso più elevati. La memoria secondaria è un tipo di memoria ad alta capacità, basso costo e non volatile. La memoria cache si posiziona ad un livello intermedio tra CPU e memorie centrali.

L'utilità della memoria cache è quella di far rimanere in memoria i dati che il processore utilizza più spesso, quando la CPU carica dei dati dalla memoria centrale, questi vengono salvati anche sulla memoria cache in modo tale che nelle operazioni successive i dati possano essere letti più velocemente rispetto alla memoria centrale.

Le memorie cache sono suddivise in due livelli, **cache di primo livello (L1)** e **cache di secondo livello (L2)**. La cache di primo livello è integrata direttamente all'interno della CPU, mentre quella di secondo livello è conosciuta come cache esterna. La memoria cache di secondo livello è composta

principalmente da SRAM, ciò garantisce una maggiore velocità in quanto non è necessario eseguire il refresh del contenuto della cella visto che quest'ultima è composta da due transistor collegati in modo da avere solo lo stato 0 o 1. La memoria cache funge da memoria di transito fra la RAM centrale e la CPU; infatti, ogni qual volta la CPU preleva informazioni dalla RAM ne viene prodotta una copia che finisce nella cache per essere immediatamente reperibile in qualsiasi accesso successivo. Nella cache vengono anche copiate le informazioni che in memoria sono adiacenti al dato appena richiesto così che nelle successive operazioni la CPU possa accedere più velocemente alle informazioni.

Principio di località:

- **Spaziale:** se nell'esecuzione di un programma la CPU accede ad una cella di memoria, è molto probabile che le celle vicine siano le prossime a cui deve accedere; pertanto, si carica un "blocco" di istruzioni seguenti così da risparmiare accessi in memoria.
- **Temporale:** se un programma ha effettuato un accesso ad una zona di memoria, è molto probabile che poco dopo acceda nuovamente a quella zona.

Per capire meglio come funziona la memoria cache introduciamo alcuni termini:

Hit:

Quantità di volte in cui la CPU legge la cache senza coinvolgere la RAM.

Miss:

Numero di volte in cui il dato non è presente nella cache e bisogna pescarlo nella RAM.

L'efficienza della cache viene misurata in hit-rate ovvero la percentuale di HIT in confronto al numero di accessi MISS.

Hit rate:

Accessi effettuati con successo / numero totale degli accessi.

Miss rate:

Accessi senza successo / numero totale degli accessi ($1 - \text{hit rate}$).

Tipologie di memoria cache:

Direct mapped: ogni indirizzo di memoria principale corrisponde esattamente a un blocco di cache, sussiste un'associazione uno a molti tra le linee di cache e linee di memoria.

Fully associative: consente di memorizzare i dati in qualsiasi blocco della cache, invece di forzare ogni indirizzo di memoria in un determinato blocco.

Set associative: è un compromesso tra cache fully associative e cache direct mapped. In una cache set associative, c'è un numero fisso di posizioni (chiamate set) in cui può essere memorizzato un dato indirizzo.

Si hanno poi i seguenti campi:

- **Tag:** contiene le informazioni necessarie a verificare se una parola della cache corrisponde o meno alla parola cercata.
- **Indice:** è necessario a selezionare il blocco della cache.
- **Bit di validità:** necessario per indicare se i dati in memoria sono validi o meno.

La dimensione della cache è di 2^n blocchi dove n bit vengono utilizzati per l'indice. La dimensione del blocco della cache è di 2^m parole, ossia 2^{m+2} byte, per cui m bit vengono utilizzati per individuare una parola all'interno di un blocco, mentre 2 bit per individuare un byte all'interno di una parola.

La dimensione del campo tag è: $32 - (n + m + 2)$

Il numero totale di bit contenuti in una cache a mappatura diretta è: $2^n * (\text{dimensioneBlocco} + \text{dimensioneTag} + \text{bitValidità})$

$$2^n * (2^m * 32 + (32 - (n + m + 2)) + 1)$$

Mappatura di un indirizzo:

Considerando una cache con 64 blocchi di 16 byte ciascuno. L'indirizzo (in byte) può essere ricavato come:

$$\text{indirizzo del blocco} = \frac{\text{indirizzo del dato in byte}}{\text{byte per blocco}}$$

Dimensione del blocco di cache:

La dimensione del blocco di cache è strettamente legata con la frequenza di miss; la frequenza di miss diminuisce con blocchi di cache grandi; tuttavia, se la dimensione del blocco è troppo grande rispetto alla dimensione della cache la frequenza di miss aumenta. La miss penalty è la differenza tra il tempo di accesso al livello inferiore di memoria e il tempo di accesso alla cache.

Sostituzione dei blocchi:

Nella cache direct mapped se il blocco di memoria p mappato in una linea di cache già occupata si elimina il contenuto precedente e lo si rimpiazza con il nuovo blocco. In caso di cache fully associative, può essere sostituito ogni blocco della cache, mentre per quanto riguarda la cache set associative è necessario scegliere tra gli N blocchi del set.

Per la sostituzione dei blocchi si possono scegliere diverse politiche:

- **Random:** la scelta del blocco avviene in maniera casuale.
- **LRU (least recently used):** il blocco sostituito sarà quello utilizzato da meno tempo, per capire quale blocco deve essere rimosso viene associato un contatore, che assume valore massimo quando si accede al blocco e viene decrementato di 1 ogni volta che l'accesso viene effettuato su un blocco differente.
- **FIFO (first in first out):** il blocco sostituito sarà quello utilizzato da più tempo, si segue un processo simile a quello utilizzato per la politica LRU.

Gestione delle miss in lettura:

Quando è necessario sostituire un elemento all'interno della cache devo gestire un accesso in scrittura. Tendendo presente che le scritture sono molto meno frequenti rispetto alle letture possiamo aspettarci delle performance migliori in lettura.

La lettura inizia quando è disponibile l'indirizzo del blocco, mentre per la scrittura è necessario aspettare la verifica del tag.

Quando cerchiamo di scrivere un dato nella cache rischiamo di generare un'incongruenza, generalmente queste situazioni si verificano quando le informazioni presenti ai livelli inferiori di

memoria non si sono ancora aggiornate, in questo frangente la CPU deve essere messa in condizione di stallo.

La scrittura all'interno della cache può essere gestita in diversi modi:

- **Write-Trough:** La scrittura del dato nella memoria cache avviene immediatamente, ciò significa che simultaneamente il dato viene scritto sia nella cache sia nella memoria a livello inferiore. Il vantaggio principale consiste nella semplicità di implementazione, questa soluzione permette anche il mantenimento della coerenza delle informazioni in quanto lavoro su entrambe le memorie. Lo svantaggio principale è la velocità con cui vengono effettuate le operazioni, che è pari alla performance più bassa.
- **Write-Back:** La scrittura del dato avviene solo nel blocco della cache, non si tratta di un'operazione simultanea, quando il blocco nella cache viene sostituito allora può essere scritto nel livello di memoria inferiore. Il vantaggio di questa soluzione è la velocità di scrittura pari a quella della cache.
- **Write-Buffer:** dispongo un buffer per la scrittura tra la cache e la memoria di livello inferiore, in questo caso i dati vengono scritti sia nella cache che nel buffer e il controller della memoria a sua volta riscrive i dati dal buffer nella memoria di livello inferiore, la scrittura è asincrona e indipendente.

Se la frequenza di scrittura è molto elevata (per esempio quando ho molte istruzioni da eseguire), il buffer potrebbe andare in overflow. Il problema può essere risolto o modificando la cache in write-back oppure si può introdurre una cache di secondo livello di tipo write back.

Write miss:

È una situazione che si verifica quando si cerca di scrivere in un blocco che non è presente nella cache, per risolvere il problema abbiamo a disposizione due modalità:

1. **Write-Allocate:** L'intero blocco viene trasferito all'interno della cache dove subisce una modifica, si effettua la scrittura con una delle strategie previste. Questo metodo viene utilizzato quando è implementato il write back.
2. **No-Write Allocate:** in questo caso la cache non viene modificata, il blocco viene scritto direttamente in memoria inferiore senza essere modificato. Questo metodo viene utilizzato quando è implementato il write trough.

Tips and tricks:

- I registri per le eccezioni sono quelli del coprocessore 0.
- Se il registro cause ha un valore espresso in esadecimale, lo trasformo in binario, Prendo i 7 bit meno significativi e rimuovo (shift left) gli ultimi due. Converto i 5 bit rimanenti in decimale e confronto il risultato con la tabella (appendice A).
- $$\text{indirizzo del blocco} = \frac{\text{IndirizzoDatoInByte}}{\text{BytePerBlocco}}$$
- $$\text{indirizzo cache} = \text{indirizzoBlocco} \% \text{numeroBlocchi}$$
- $$\text{CPI}_m = \text{CPI} * \text{HitProbability} + \text{MissPenalty}(1 - \text{HitProbability})$$
- $$\text{Velocità}_m = \frac{\text{frequenzaInHz}}{\text{CPI}_m} \text{ espressa in } \frac{\text{Istruzioni}}{\text{secondi}}$$
- $$\text{NumeroBloccoMappatoInMemoriaPrincipale} = \text{IndirizzoBlocco} \% \text{NumeroBlocchi}$$
- Il gestore delle eccezioni può essere collocato nello spazio **kernel text** a partire dalla locazione 0x80000180

- Se il registro cause assume il valore 0x00000000 non si verifica alcuna eccezione.
- *Tempo di trasferimento* = *byteDaTrasmettere* / *bandaPassante*